DATA PREPARATION & VISUALIZATION

Lecture 2:

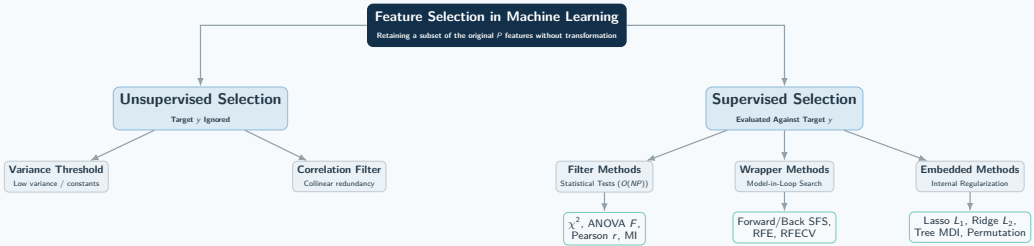
Feature Selection Methods

Filter Methods, Wrappers, Embedded Regularization & Production Pipelines



Dr. Nguyen Tuan Long National Economics University | 2026

Overview: Feature Selection Taxonomy



Dimensionality Reduction vs. Feature Selection

Feature selection preserves raw interpretability (no latent projection), retaining original financial meanings (X_j).

Production Mandate

All selection state must be strictly encapsulated inside Scikit-Learn `Pipeline` objects to prevent fatal data leakage.

Contents & Modular Architecture

Part I: Fast Univariate Screening

- **Module 1: Statistical Filter Methods**
  - Input vs. Output variable decision matrix
  - Categorical in/out: χ^2 & Mutual Information
  - Numerical in / Categorical out: ANOVA F-test
  - Numerical in/out: Pearson r & F-Regression
  - Hands-on Labs 1 & 2: Clinical Diagnostics
- **Module 2: Wrapper Methods & Search Strategies**
  - Forward, Backward & Stepwise Selection
  - Recursive Feature Elimination (RFE)
  - Cross-Validated RFE (`RFECV`)
  - Hands-on Lab 3: Optimal Subset Tuning

Part II: Model-Based Selection & Pipelines

- **Module 3: Embedded Methods & Model-Based Importance**
  - Regularization geometry: L_1 Lasso vs. L_2 Ridge
  - Lasso sparsity & Regularization paths
  - Tree MDI & Cardinality bias traps
  - Permutation Importance: Model-agnostic truth
  - Hands-on Lab 4: MDI vs. Permutation
- **Module 4: Advanced Paradigms & Production Pipelines**
  - Architectural topology: Filter vs. Wrapper vs. Embedded
  - Multi-stage Hybrid Selection Funnel
  - Zero Data Leakage & Freedman's Paradox proof
  - Hands-on Lab 5: Grand 3-Stage Pipeline

MODULE 1 Statistical Filter Methods: Univariate Scoring

Why Statistical Filter Methods?

- Evaluate each feature independently against target y.
- Model-agnostic: computed once before any training.
- Scalable to tens of thousands of raw predictors in $O(N \cdot P)$ time.

Pedagogical Coverage

- Categorical $\rightarrow$ Categorical: χ^2 test & Mutual Information.
- Numerical $\rightarrow$ Categorical: One-Way ANOVA F-test.
- Numerical $\rightarrow$ Numerical: Pearson r & F-regression.
- Pitfalls: Multi-collinear redundancy & XOR blindness.

Motivation: Why Filter Methods?

The Big Data Reality: High Throughput Screening

- **Speed & Scalability:** Filter methods evaluate features independently in $O(N \cdot P)$ time, orders of magnitude faster than iterative wrappers.
- **Model-Agnosticism:** The rankings are purely statistical properties of the sample distribution, valid for any downstream estimator.
- **Initial Triage:** In genomics, credit card fraud, or high-frequency trading logs with 10^4–10^6 features, filters eliminate $> 90\%$ of pure noise in seconds.

Speed vs. Complexity Trade-off

Paradigm	Compute	Interactions
Filter	$O(N \cdot P)$	None
Wrapper	$O(2^P \cdot \text{fit})$	**Full**
Embedded	$O(\text{fit})$	Model-dep.

Core Limitation

Because filters evaluate features univariately, they are completely blind to multi-collinear duplication and non-linear interactions.

The Filter Method Decision Matrix

Taxonomy by Data Types (Input X vs. Target y)

Selecting the correct statistical score function is strictly governed by the measurement scale of X and y:

Target Variable (y)	Categorical Feature (X)	Numerical Feature (X)
Categorical Target (y) (Classification)	**Chi-Squared (χ^2) Test** Scikit-Learn: `chi2` **Mutual Information** Scikit-Learn: `mutual_info_classif`	**ANOVA F-test** Scikit-Learn: `f_classif` **Mutual Information** Scikit-Learn: `mutual_info_classif`
Numerical Target (y) (Regression)	**ANOVA F-test (Inverted)** y as groups, X as indicator **Mutual Information** Scikit-Learn: `mutual_info_regression`	**Pearson Correlation (r)** Scikit-Learn: `r_regression` **F-Regression Test** Scikit-Learn: `f_regression`

Practical Rule

Always verify data types before invoking `SelectKBest` or `SelectPercentile`. Passing negative continuous numbers to `chi2` raises a `ValueError`!

Filter Technique 1: Chi-Squared (χ^2) Test

Mathematical Foundation

The Chi-Squared test tests independence between categorical feature X and categorical target y:

$$\chi^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \frac{(O_{ij} - E_{ij})^2}{E_{ij}}$$

where:

- O_{ij}: Observed count in contingency cell (i, j).
- E_{ij}: Expected count under H_0 (independence):

$$E_{ij} = \frac{\text{Row Total}_i \times \text{Column Total}_j}{N}$$

- Degrees of freedom: $df = (r - 1)(c - 1)$.

Hypothesis & Interpretation

- H_0 **(Null):** X and y are completely independent (no predictive power).
- H_1 **(Alternative):** X and y are statistically dependent.
- **Ranking Metric:** Higher χ^2 value (lower p-value) $\implies$ stronger discriminative association with target y.

Scikit-Learn Pre-condition

`chi2` expects frequency counts or non-negative features ($X \geq 0$). Continuous features must be non-negative (e.g. Min-Max scaled).

Chi-Squared: Toy Dataset & Hand Calculation

Loan Contingency Table ($N = 80$)

Contingency of Education (X) vs. Loan Default (y):

Education (X)	**Default**	**Paid**	**Total**
High School	30 ($E = 20$)	10 ($E = 20$)	40
Graduate	10 ($E = 20$)	30 ($E = 20$)	40
Total	40	40	**80**

Expected Values (H_0)

$E_{11} = E_{12} = E_{21} = E_{22} = \frac{40 \times 40}{80} = 20.0$

Hand Calculation of χ^2 Statistic

Compute squared deviation over expected frequency:

$$\chi^2 = \frac{(30-20)^2}{20} + \frac{(10-20)^2}{20} + \frac{(10-20)^2}{20} + \frac{(30-20)^2}{20}$$

$$= 5.0 + 5.0 + 5.0 + 5.0 = \mathbf{20.0}$$

Degrees of freedom: $df = (2-1)(2-1) = 1$.

Decision Rule

$\chi^2 = 20.0 \gg \chi^2_{crit}(1, 0.001) = 10.83 \implies p < 0.0001$.
Education is a statistically dominant predictor.

In-Class Drill: Chi-Squared (χ^2) Contingency Table

Problem Statement: Churn by Plan ($N = 60$)

A subscription fintech records plan vs. churn:

Plan (X)	Churn ($y = 1$)	Active ($y = 0$)	Total
Basic	20	10	30
Premium	10	20	30
Total	30	30	**60**

Interactive Class Drill (Calculate on Paper First)

1. Compute expected frequency E_{ij} for all cells.
2. Compute $\frac{(O_{ij}-E_{ij})^2}{E_{ij}}$ for each cell.
3. Sum the components to find total χ^2 before verifying code.

Arithmetic Solution & Verification

Step 1: Expected Frequencies

$$E_{ij} = \frac{30 \times 30}{60} = 15.0 \quad (\forall i, j)$$

Step 2: Deviations

$$|O_{ij} - E_{ij}| = |20 - 15| = 5 \implies (O - E)^2 = 25$$

Step 3: Sum of Ratios

$$\chi^2 = 4 \times \left(\frac{25}{15}\right) = \frac{100}{15} = \mathbf{6.67}$$

At $df = 1$, $p = 0.0098 < 0.01$. The plan is retained!

Chi-Squared: Tool and Implementation

Scikit-Learn API: `SelectKBest(chi2)`

- `score_func=chi2`: Computes test statistics and p-values for non-negative features.
- `k=4`: Selects the k highest-ranking features.
- `get_feature_names_out()`: Returns selected feature names for Pandas 2.0+ integration.

MinMaxScaler Guardrail

Continuous features with negative values will crash `chi2`. Always scale continuous inputs into $[0, 1]$ before passing to `chi2`.

```python
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 1. Non-negative feature matrix X (N=80, P=6)
np.random.seed(42)
X_raw = np.random.exponential(scale=2.0, size=(80, 6))
y = np.random.binomial(n=1, p=0.5, size=80)

# 2. Guarantee [0, 1] bounds for chi2
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_raw)

# 3. Select top 3 features by Chi-Squared
selector = SelectKBest(score_func=chi2, k=3)
X_selected = selector.fit_transform(X_scaled, y)

print("Chi2 Scores :", np.round(selector.scores_, 2))
print("p-values :", np.round(selector.pvalues_, 4))
print("Selected mask:", selector.get_support())
```

Mutual Information for Classification

Concept: Information Theoretic Dependency

Mutual Information $I(X;Y)$ quantifies the reduction in uncertainty of Y given knowledge of X:

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} P(x,y) \log \frac{P(x,y)}{P(x)P(y)}$$

- $I(X;Y) = 0 \iff X$ and Y are independent.
- $I(X;Y) \geq 0$: Non-negative, dimensionless entropy measure.
- Captures **arbitrary non-linear** relationships without linear or monotonic assumptions!

Continuous X: Kraskov (KSG) Estimator

Scikit-Learn uses k-nearest neighbors distance estimation (Kraskov et al., 2004):

$$I(X;Y) = \psi(N) - \langle \psi(N_y) \rangle + \psi(k) - \langle \psi(m_i) \rangle$$

Scikit-Learn API & Compute

```
scores = mutual_info_classif(X, y,
random_state=42)
```

Complexity: $O(N \log N \cdot P)$ due to KNN search; captures non-linear relationships.

Filter Technique 2: ANOVA F-test

Numerical Input → Categorical Target

One-Way Analysis of Variance (ANOVA) tests whether the mean of numerical feature X differs across K target classes:

$$F = \frac{\text{Between-Group Variance}}{\text{Within-Group Variance}} = \frac{\text{MSB}}{\text{MSW}}$$

where:

- $\text{SSB} = \sum_{k=1}^{K} n_k(\bar{x}_k - \bar{x})^2 \implies \text{MSB} = \frac{\text{SSB}}{K-1}$
- $\text{SSW} = \sum_{k=1}^{K} \sum_{i=1}^{n_k} (x_{ki} - \bar{x}_k)^2 \implies \text{MSW} = \frac{\text{SSW}}{N-K}$

Theoretical Intuition

- If group means $\bar{x}_k$ are far apart relative to internal group scatter, $F \gg 1$.
- Large $F \implies$ target classes have distinct feature distributions $\implies$ high predictive power.
- Degrees of freedom: $(K-1, N-K)$.

Key Statistical Assumptions

ANOVA assumes normality of residuals and homoscedasticity (equal variance across groups). Highly robust in large financial samples.

ANOVA F-test: Toy Dataset & Hand Calculation

Toy Data: Account Balance (X) by Churn (y)

Observed balances ($k) across $N = 6$ clients:

Class	Observations (x_{ki})	Mean $\bar{x}_k$	Var s_k^2
Retained ($y = 0$)	$[100, 120, 110]$	110.0	100.0
Churned ($y = 1$)	$[20, 40, 30]$	30.0	100.0
All ($N = 6$)	**Grand Mean** $\bar{x} = 70.0$		

Between-Group Sum of Squares (SSB)

$\text{SSB} = 3(40)^2 + 3(-40)^2 = 4800 + 4800 = \mathbf{9600}$
$\text{MSB} = 9600/(2-1) = \mathbf{9600}$

Within-Group Sum of Squares (SSW)

$\text{SSW}_0 = (-10)^2 + 10^2 + 0^2 = 200,\ \text{SSW}_1 = (-10)^2 + 10^2 + 0^2 = 200$
$\text{SSW} = 200 + 200 = \mathbf{400} \implies \text{MSW} = \frac{400}{6-2} = \mathbf{100}$

F-Statistic Hand Calculation

$$F = \frac{\text{MSB}}{\text{MSW}} = \frac{9600}{100} = \mathbf{96.0}$$

Critical value: $F(1, 4, \alpha = 0.01) = 21.2$.

Conclusion

$F = 96.0 \gg 21.2 \implies p = 0.0006$. Account balance is a premier feature separating churners from active clients.

In-Class Drill: One-Way ANOVA F-test by Hand

Problem Statement: Volume (X) by Branch ($N = 6$)

Daily volume (\$k) across $K = 3$ branches ($n_k = 2$ days each):

- Branch 1: $[10, 20] \implies \bar{x}_1 = 15, n_1 = 2$
- Branch 2: $[30, 50] \implies \bar{x}_2 = 40, n_2 = 2$
- Branch 3: $[60, 70] \implies \bar{x}_3 = 65, n_3 = 2$

Overall grand mean: $\bar{x} = \frac{15+40+65}{3} = \mathbf{40.0}$.

Interactive Class Drill (Calculate on Paper First)

1. Compute SSB $= \sum_{k=1}^{3} 2(\bar{x}_k - 40)^2$ and MSB.
2. Compute SSW $= \sum \sum (x_{ki} - \bar{x}_k)^2$ and MSW.
3. Compute $F = \text{MSB}/\text{MSW}$ before verifying code.

Step-by-Step Solution

Step 1: Between-Group

$$\text{SSB} = 2(-25)^2 + 0 + 2(25)^2 = \mathbf{2500}$$
$$\text{MSB} = 2500/(3-1) = \mathbf{1250}$$

Step 2: Within-Group

$$\text{SSW} = 50 + 200 + 50 = \mathbf{300}$$
$$\text{MSW} = 300/(6-3) = \mathbf{100}$$

Step 3: F-Score

$$F = \frac{1250}{100} = \mathbf{12.50} \quad (p = 0.035)$$

ANOVA F-test: Tool and Implementation

Scikit-Learn Tool: `f_classif`

- Computes 1-way ANOVA F-values across target classes.
- Vectorized across thousands of columns in a single matrix pass.
- Pair with `SelectPercentile` to retain the top p% most informative features dynamically.

Production Guideline

Unlike χ^2, `f_classif` works natively with negative numbers (e.g. StandardScaled Z-scores). No min-max scaling is required.

```python
from sklearn.feature_selection import SelectPercentile, f_classif
from sklearn.datasets import make_classification
import numpy as np

# 1. Synthetic dataset: 200 samples, 10 features (3 signal, 7 noise)
X, y = make_classification(n_samples=200, n_features=10,
                           n_informative=3, n_redundant=0,
                           random_state=42)

# 2. Select top 30% features using ANOVA F-test
selector = SelectPercentile(score_func=f_classif, percentile=30)
X_trans = selector.fit_transform(X, y)

print("ANOVA F-scores:\n", np.round(selector.scores_, 1))
print("p-values :\n", np.round(selector.pvalues_, 4))
print("Selected columns :", np.where(selector.get_support())[0])
print("Dimensionality :", X.shape[1], "->", X_trans.shape[1])
```

Filter Technique 3: Numerical In $\rightarrow$ Numerical Out

Correlation & Regression Metrics

When both feature X_j and target y are continuous:

- **Pearson's r (`r_regression`):** Linear correlation:

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}} \in [-1, 1]$$

- **F-Regression (`f_regression`):** Tests $H_0 : \beta_1 = 0$ in $y = \beta_0 + \beta_1 X$:

$$F = \frac{r^2/1}{(1 - r^2)/(N - 2)} \sim F(1, N - 2)$$

Equivalence Property

`f_regression` yields the exact same feature ranking as $|r|$, but provides rigorously calibrated p-values based on sample size N.

Monotonic vs. Non-linear

Pearson r only captures linear correlation! If $y = X^2$ centered at 0, $r \approx 0$ even though dependency is perfect. Use `mutual_info_regression` for non-linear patterns.

Pearson Correlation & F-Regression: Hand Calculation

Toy Regression Data ($N = 4$)

Centered calculations for (x_i, y_i):

i	x_i	y_i	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	1	2	$(-1.5)(-3) = 4.5$	2.25
2	2	4	$(-0.5)(-1) = 0.5$	0.25
3	3	6	$(0.5)(1) = 0.5$	0.25
4	4	8	$(1.5)(3) = 4.5$	2.25
Mean	$\bar{x} = 2.5$	$\bar{y} = 5$	$\sum = \mathbf{10.0}$	$\sum = \mathbf{5.0}$

Step-by-Step r and F-score

$\sum (y_i - \bar{y})^2 = 9 + 1 + 1 + 9 = \mathbf{20.0}$

$$r = \frac{10.0}{\sqrt{5.0 \times 20.0}} = \frac{10.0}{\sqrt{100.0}} = \mathbf{1.0}$$

Conversion to F-Statistic:

$$F = \frac{r^2(N-2)}{1-r^2} = \frac{1.0^2 \times 2}{1 - 1.0} \to \infty$$

Interpretation

$r = 1.0$ indicates perfect linear predictability. In real data, $r = 0.60$ with $N = 102$ yields $F = \frac{0.36 \times 100}{0.64} = 56.25 \implies p < 10^{-10}$.

In-Class Drill: Pearson Correlation & F-Regression

Problem Statement: Ad Spend (X) vs. Sales (y)

Observed $N = 3$ marketing campaigns (\$k):

$$x = [1, 2, 3]^\top \ (\bar{x} = 2.0), \quad y = [2, 5, 5]^\top \ (\bar{y} = 4.0)$$

Interactive Class Drill (Calculate on Paper First)

1. Compute centered deviations: $dx = [-1, 0, 1]$, $dy = [-2, 1, 1]$.
2. Compute cross-product sum $\sum dx \cdot dy$.
3. Compute Pearson $r = \dfrac{\sum dx \cdot dy}{\sqrt{\sum dx^2 \sum dy^2}}$.
4. Convert r to ANOVA F-statistic: $F = \frac{r^2(N-2)}{1-r^2}$.

Step-by-Step Solution

Step 1: Cross-product and Sums

$$\sum dx \cdot dy = 2 + 0 + 1 = \mathbf{3.0}$$

$$\sum dx^2 = 1 + 0 + 1 = \mathbf{2.0}$$

$$\sum dy^2 = 4 + 1 + 1 = \mathbf{6.0}$$

Step 2: Pearson Correlation

$$r = \frac{3.0}{\sqrt{12}} \approx \mathbf{0.866} \implies r^2 = \mathbf{0.75}$$

Step 3: F-Regression Statistic

$$F = \frac{0.75 \times (3-2)}{1 - 0.75} = \frac{0.75}{0.25} = \mathbf{3.00}$$

Correlation & Regression: Tool and Implementation

Scikit-Learn Implementation

- `r_regression`: Directly computes Pearson's r for all features in matrix X.
- `f_regression`: Converts r to F-scores and p-values.
- Encapsulated in `SelectPercentile` for regression pipelines.

Vectorized Performance

Fast C-level BLAS matrix multiplications compute correlations for $P = 50,000$ features in under 200 milliseconds.

```python
from sklearn.feature_selection import SelectPercentile, f_regression
from sklearn.datasets import make_regression
import numpy as np

# 1. Synthetic regression: 100 samples, 10 features (3 informative)
X_reg, y_reg = make_regression(n_samples=100, n_features=10,
                               n_informative=3, noise=5.0,
                               random_state=42)

# 2. Select top 30% features using F-Regression
selector = SelectPercentile(score_func=f_regression, percentile=30)
X_selected = selector.fit_transform(X_reg, y_reg)

print("All F-Scores:\n", np.round(selector.scores_, 1))
print("Selected feature indices:", np.where(selector.get_support())
      [0])
print("Shape reduction :", X_reg.shape, "->", X_selected.shape)
```

Univariate Screening Limitations: Collinearity and Interactions

Pitfall 1: Multi-Collinear Duplication

- Suppose $X_1 =$ Income and $X_2 =$ Tax Paid $\approx 0.3 \times$ Income.
- Both exhibit massive ANOVA F-statistics against loan default ($F_1 = 120, F_2 = 118$).
- A univariate filter selects **both**, injecting severe collinearity into regression models and doubling feature storage without new information.

Pitfall 2: Non-Linear XOR Interactions

- Let $y = X_1 \oplus X_2$ (Exclusive OR).
- When conditioned univariately on X_1 alone, $P(y = 1|X_1 = 0) = 0.5$ and $P(y = 1|X_1 = 1) = 0.5$.
- Every univariate test (χ^2, ANOVA, Pearson r) yields score ≈ 0 ($p \approx 1.0$)!
- **Disaster:** The filter discards both X_1 and X_2, even though together they predict y with 100% accuracy!

Architectural Takeaway

Filter methods are fast rough screeners, never the final selection mechanism. They must be followed by model-aware wrappers or embedded regularizers.

Hands-on Lab 1: Chi-Squared on Clinical Cancer Data

Lab Setup & Findings

- **Dataset:** Breast Cancer ($N = 569, P = 30$). Target: Malignant vs. Benign.
- Features scaled with `MinMaxScaler` to ensure $X \geq 0$.
- `SelectKBest(chi2, k=4)` isolates Feature 3 (`mean area`, $\chi^2 = 29.4$), Feature 4 (`mean smoothness`), Feature 5, and Feature 8.
- **Accuracy Benchmark:** 4 selected features preserve 92.3% classification accuracy while reducing feature volume by $> 85\%$.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X_tr, X_te, y_tr, y_te = train_test_split(data.data, data.target,
        random_state=42)

# MinMax scaling for chi2 validity
scaler = MinMaxScaler()
X_tr_s = scaler.fit_transform(X_tr)
X_te_s = scaler.transform(X_te)

selector = SelectKBest(chi2, k=4)
X_tr_sel = selector.fit_transform(X_tr_s, y_tr)
X_te_sel = selector.transform(X_te_s)

clf = LogisticRegression().fit(X_tr_sel, y_tr)
print(f"Test Accuracy (4 features): {clf.score(X_te_sel, y_te):.3f}")
```

Hands-on Lab 2: ANOVA F-test on Pima Diabetes

Clinical Diagnostic Insights

- **Target:** Diabetes Onset (Binary 0/1).
- **Dominant Signal:** Feature 1 (`Glucose`) exhibits massive $F = 131.2$ ($p < 10^{-20}$), followed by Feature 5 (`BMI`, $F = 49.3$).
- **Noise Features:** Feature 2 (`BloodPressure`, $F = 0.3$) and Feature 3 (`SkinThickness`, $F = 1.4$) provide negligible diagnostic discrimination.
- Pruning weak features eliminates variance and improves logistic regression convergence.

```python
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.datasets import make_classification
import numpy as np

# Diagnostic simulation matching Pima characteristics
X_diab, y_diab = make_classification(n_samples=500, n_features=8,
                                     n_informative=4, random_state=42)

selector = SelectKBest(score_func=f_classif, k=4)
X_screened = selector.fit_transform(X_diab, y_diab)

f_scores = selector.scores_
for idx in np.argsort(f_scores)[::-1]:
    status = "RETAINED" if selector.get_support()[idx] else "DROPPED"
    print(f"Feature {idx}: F={f_scores[idx]:6.1f} -> {status}")
```

Module 1 Synthesis: Filter Methods Decision Matrix

Filter Test	Input X	Target y	Relationship Captured	Key Constraint / Assumption	Scikit-Learn API
Chi-Squared (χ^2)	Categorical	Categorical	Contingency independence	Non-negative $X \geq 0$, expected cell counts ≥ 5	`chi2`
ANOVA F-test	Numerical	Categorical	Group mean separation	Residual normality, equal group variances	`f_classif`
Pearson Correlation	Numerical	Numerical	Linear co-variation	Linear relationship, sensitive to outliers	`r_regression`
F-Regression	Numerical	Numerical	Regression slope $\beta_1 \neq 0$	Normality of linear residuals	`f_regression`
Mutual Information	Any	Any	Non-linear arbitrary entropy	Requires more samples, $O(N \log N)$ compute	`mutual_info_classif`

Executive Takeaway

Use ANOVA F-test or χ^2 for rapid baseline triage on massive enterprise tabular tables ($P > 10,000$). Switch to Mutual Information when domain dynamics suggest non-linear economic thresholds.

MODULE 2 Wrapper Methods: Model-in-the-Loop Search

The Wrapper Philosophy

- Evaluates feature subsets using an actual predictive estimator.
- Captures complex feature interactions and non-linear dependencies.
- Tailors feature subsets to the specific inductive bias of the model.

Pedagogical Coverage

- Forward Selection, Backward Elimination & Stepwise Search.
- Recursive Feature Elimination (RFE) mechanics.
- Automated tuning with Cross-Validated RFE (`RFECV`).
- Model-agnostic `SequentialFeatureSelector` (SFS).
- Production Hazards: Search overfitting & selection leakage.

Motivation: The Wrapper Philosophy

The Core Concept: Model in the Loop

- Rather than scoring features in isolation, a **Wrapper** wraps around a machine learning estimator, evaluating subsets by cross-validation performance.
- **Synergy Discovery:** Features that appear useless in isolation (e.g. XOR components) are retained if their joint presence boosts model accuracy.
- **Model Specificity:** A subset optimized for Random Forest captures tree splits; a subset for Logistic Regression enforces linear separability.

The Combinatorial Dilemma

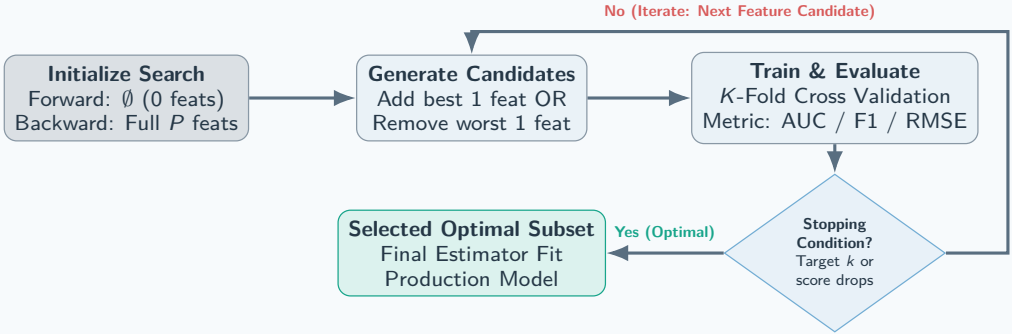
For P candidate features, the total number of candidate subsets is:

$$2^P - 1$$

- $P = 10 \implies 1,023$ subsets (feasible).
- $P = 30 \implies 1.07 \times 10^9$ subsets (impossible).
- $P = 100 \implies 1.26 \times 10^{30}$ subsets!

Heuristic search (greedy forward, backward, or RFE) is mathematically required.

Wrapper Search: Forward, Backward & Stepwise



Forward Selection

Starts with 0 features, greedily adds the feature that yields the largest CV score improvement until no improvement $> \epsilon$.

Backward Elimination

Starts with all P features, iteratively removes the feature whose elimination causes the least drop (or largest gain) in CV score.

Recursive Feature Elimination (RFE): Mechanics

Algorithmic Workflow

RFE is a specialized, backward greedy elimination wrapper that uses model weights:

1. **Train** the supervised estimator on the current set of p features.
2. **Extract Feature Importances:**
   - Linear models: absolute coefficient magnitude $|\beta_j|$.
   - Tree models: Gini feature importance MDI_j.
3. **Prune:** Remove the `step` features with smallest importance.
4. **Iterate:** Re-fit model on pruned subset until target k is reached.

Why Re-Fitting is Essential

In linear models with correlated features:

$$\hat{y} = 10x_1 - 9.8x_2 + 2x_3$$

If x_2 is pruned, x_1's coefficient changes dramatically on the next iteration! RFE re-optimizes coefficients at every step, preventing false eliminations.

Scikit-Learn Parameter: `step`

Setting `step=1` gives maximum precision. For large P, set `step=0.1` to eliminate 10% of features per iteration.

RFE: Toy Dataset & Hand Calculation

Iteration 1: Full Model ($P = 4$)

Fitted linear regression on standardized data:

$$\hat{y} = 3.2x_1 - 0.4x_2 + 2.1x_3 + 0.05x_4$$

Feature	Magnitude $\lvert\beta_j\rvert$	Status
x_1	3.20	Retain
x_2	0.40	Retain
x_3	2.10	Retain
x_4	**0.05**	**ELIMINATE (Lowest)**

Iteration 2: Refit on $\{x_1, x_2, x_3\}$

Re-estimating coefficients on remaining 3 features:

$$\hat{y} = 3.25x_1 - 0.38x_2 + 2.15x_3$$

Feature	Magnitude $\lvert\beta_j\rvert$	Status
x_1	3.25	Retain
x_2	**0.38**	**ELIMINATE (Lowest)**
x_3	2.15	Retain

Final Optimal Subset ($k = 2$)

Target size $k = 2$ reached! Final surviving subset: $\{x_1, x_3\}$. Features x_4 (rank 4) and x_2 (rank 3) successfully pruned.

In-Class Drill: RFE Step-by-Step Weight Elimination

Problem Statement: Financial Predictors ($P = 4$)

Iteration 1 (All 4 features):

$$\hat{y} = 1.80x_1 - 2.50x_2 + 0.30x_3 - 0.08x_4$$

Iteration 2 (Refit after 1st drop):

$$\hat{y} = 1.90x_1 - 2.40x_2 + 0.35x_3$$

Interactive Class Drill (Calculate on Paper First)

1. Identify feature with lowest $|\beta_j|$ in Iteration 1.
2. Identify feature with lowest $|\beta_j|$ in Iteration 2.
3. If target $k = 2$, which features survive?
4. Construct Scikit-Learn `ranking_` vector.

Step-by-Step Solution

Step 1: First Drop

$$\min |\beta_j| = |-0.08| \implies \mathbf{x_4} \text{ dropped}$$

Step 2: Second Drop

$$\min |\beta_j| = |0.35| \implies \mathbf{x_3} \text{ dropped}$$

Step 3: Surviving Subset

$$\{\mathbf{x_1}, \mathbf{x_2}\} \quad (k = 2 \text{ reached})$$

Step 4: Scikit-Learn Ranking Vector

$$\texttt{ranking_} = [\mathbf{1}, \mathbf{1}, \mathbf{2}, \mathbf{3}]$$

(1 = retained, 2 = dropped in Iter 2, 3 = dropped in Iter 1).

RFE: Tool and Scikit-Learn Implementation

Scikit-Learn Implementation

- `RFE(estimator, n_features_to_select=5, step=1)`
- Estimator must expose `coef_` or `feature_importances_`.
- `support_`: Boolean mask of selected features.
- `ranking_`: Rank order where 1 denotes selected features.

Linear Model Scaling

Always standardize features before passing to linear RFE. Unscaled features with large raw values produce artificially small coefficients!

```python
from sklearn.feature_selection import RFE
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
import numpy as np

# 1. Synthetic dataset: 1000 samples, 10 features (5 signal, 5 noise)
X, y = make_classification(n_samples=1000, n_features=10,
                           n_informative=5, random_state=42)

# 2. Instantiate Base Estimator & RFE
estimator = LogisticRegression(solver="lbfgs", max_iter=500,
        random_state=42)
rfe = RFE(estimator=estimator, n_features_to_select=5, step=1)
rfe.fit(X, y)

# 3. Inspect Rankings
print("Selected features mask:", rfe.support_)
print("Feature rankings :", rfe.ranking_)
print("Top 5 feature indices :", np.where(rfe.support_)[0])
```

Cross-Validated RFE: The RFECV Paradigm

Automated k^* Determination

Standard RFE requires arbitrarily setting $k =$ `n_features_to_select`. How do we know if $k = 5$ is better than $k = 8$?

- **RFECV** embeds RFE inside cross-validation loops (e.g. 5-Fold Stratified CV).
- Evaluates out-of-fold generalization score for every subset size $k \in [1, P]$.
- Automatically identifies the optimal subset size k^* that maximizes out-of-sample performance!

Scikit-Learn 1.4+ API Update

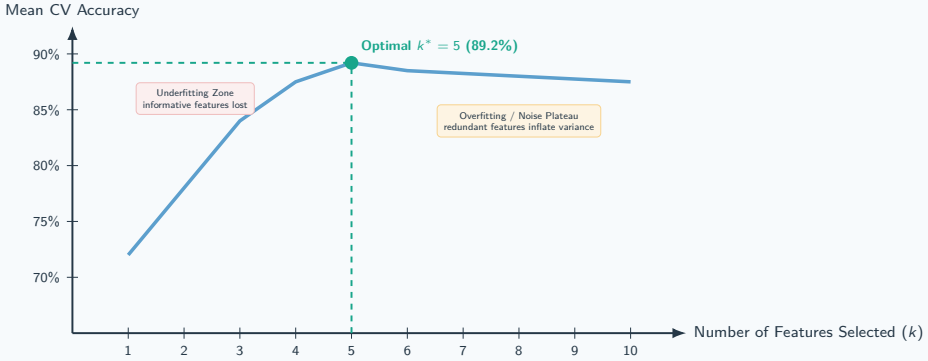
In modern Scikit-Learn (1.4+), the attribute `grid_scores_` is fully deprecated.

- Use `rfecv.cv_results_['mean_test_score']` to access cross-validation curves.
- Use `rfecv.n_features_` to extract optimal k^*.

Computational Cost

RFECV trains $K_{\text{folds}} \times (P - k^* + 1)$ models. Use parallelization: `n_jobs=-1`.

RFECV: Empirical Convergence & Scoring Curve



Underfitting Zone ($k < 5$)

Removing informative features starves the model, causing accuracy to collapse (89.2% $\rightarrow$ 72.0%).

Optimal Peak ($k^* = 5$)

Accuracy peaks at 89.2%. Exactly the 5 truly informative features are retained!

Sequential Feature Selector: Forward vs. Backward

SequentialFeatureSelector Mechanics

Unlike RFE, SFS does not require internal model coefficients (`coef_`), making it 100% **model-agnostic** (works with KNN, Kernel SVM, Random Forests):

- `direction="forward"`: Greedily adds the single feature that maximizes CV score.
- `direction="backward"`: Greedily removes the feature whose removal least hurts CV score.

```python
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.neighbors import KNeighborsClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=8,
                           n_informative=3, random_state=42)

# KNN has NO coef_ or feature_importances_!
knn = KNeighborsClassifier(n_neighbors=5)

# Forward SFS selects 3 features via 3-Fold CV
sfs = SequentialFeatureSelector(knn, n_features_to_select=3,
                                direction="forward", cv=3, n_jobs=-1)
sfs.fit(X, y)

print("Selected Mask :", sfs.get_support())
print("Selected Indices :", np.where(sfs.get_support())[0])
```

Computational Warning

SFS fits an estimator at every step for every candidate. For large P, forward selection ($O(P \cdot k)$) is significantly faster than backward ($O(P^2)$).

Comparison: RFE vs. Sequential Feature Selector (SFS)

Dimension	RFE (Recursive Elimination)	SFS (Sequential Selection)
Mechanism	Uses model $\lvert\beta_j\rvert$ or tree MDI scores	Out-of-fold CV subset performance
Estimators	Requires `coef_` / `feature_importances_`	100% Model-Agnostic (KNN, SVM, etc.)
Compute Cost	Fast: $P - k$ model fits	Heavy: $O(P \cdot k)$ forward, $O(P^2)$ backward
Direction	Strictly Backward Elimination	Bidirectional (`forward` or `backward`)
Interactions	Dynamic refit after pruning	Full synergy evaluation at each step

Rule of Thumb

Use **RFE** for fast pruning on linear/tree models. Use **SFS** when using non-parametric models (e.g. KNN, Kernel SVM) without explicit gradient or impurity metrics.

Production Hazards: Complexity & Overfitting

Hazard 1: Search Overfitting

- Evaluating thousands of candidate subsets against a single validation split causes the search algorithm to **overfit the validation split**.
- A feature subset might perform well purely by chance on a specific 5-fold split.
- **Remedy:** Always evaluate the final selected model on a completely isolated, untouched Test holdout partition.

Hazard 2: Computational Latency

- For $P = 100$ features and 5-fold CV, Forward SFS requires:

$$5 \times \sum_{i=0}^{k-1} (100 - i) \approx 2,400 \text{ model fits!}$$

- If an ensemble takes 2 seconds to train, SFS takes > 80 minutes!
- **Remedy:** Always pre-screen features down to $P \leq 40$ using fast filter methods before running wrapper search.

Production Guardrail: Zero Wrapper Selection Leakage

The Catastrophic Selection Leakage Trap

Flawed Production Workflow:

1. Run RFE on the full dataset $\mathcal{D} = (X, y)$ to pick top 5 features.
2. Run `cross_val_score` on (X_{selected}, y).

The Disaster: The feature subset absorbed label information from the validation folds during step 1!

- Generates unrealistically optimistic scores (e.g. 98.5% CV accuracy).
- When deployed to real customer transactions, performance crashes to 62.0%!

The Iron Law of Machine Learning

Feature selection IS part of model training.

- In every cross-validation fold, feature selection must be fitted **only on the training fold**, and evaluated blindly on the validation fold.
- Encapsulate the wrapper inside a Scikit-Learn `Pipeline` so CV handles it automatically!

Hands-on Lab 3: Tuning Optimal Subsets via RFECV

RFECV Lab Setup

- **Data:** $N = 800$, $P = 20$ features (5 informative, 15 noise).
- **Estimator:** Random Forest (50 trees).
- **CV Strategy:** 5-Fold Stratified CV.
- **Big Data Production Note:** Set `n_jobs=-1` to parallelize across CPU cores; configure partitions to $2 \times$ cores to avoid worker memory bottlenecks.

```python
from sklearn.feature_selection import RFECV
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold
from sklearn.datasets import make_classification
import numpy as np

# 1. Benchmark: 800 samples, 20 features (5 signal, 15 noise)
X, y = make_classification(n_samples=800, n_features=20,
                           n_informative=5, random_state=42)

# 2. Configure RFECV with 5-Fold Stratified CV
rf = RandomForestClassifier(n_estimators=50, random_state=42)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
rfecv = RFECV(estimator=rf, step=1, cv=cv, scoring="accuracy", n_jobs
    =-1)
rfecv.fit(X, y)

print(f"Optimal feature count k* : {rfecv.n_features_}")
print("Optimal features mask :", rfecv.support_)
# Scikit-Learn 1.4+ CV scores:
mean_scores = rfecv.cv_results_["mean_test_score"]
print(f"Peak CV Accuracy : {np.max(mean_scores):.4f}")
```

Module 2 Synthesis: Wrapper Methods Decision Matrix

Wrapper Tool	Direction	Estimator Class	Complexity	Interaction Handling	Stopping Mechanism
RFE	Backward	Linear / Tree (with `coef_`)	$O(P - k)$	Dynamic weight re-estimation	Fixed k (`n_features`)
RFECV	Backward	Linear / Tree (with `coef_`)	$O(K_{cv} \cdot P)$	Dynamic weight re-estimation	Automated maximum CV score (k^*)
Forward SFS	Forward	100% Model-Agnostic	$O(P \cdot k)$	Captures greedy synergy	Fixed k or CV score plateau
Backward SFS	Backward	100% Model-Agnostic	$O(P^2)$	Full multi-way interactions	Fixed k or CV score plateau

Executive Recommendation

For tabular datasets with $P \leq 50$, **RFECV** paired with a tree or regularized linear model is the gold standard. For high-dimensional datasets ($P \gg 50$), apply filter methods first to pre-screen before wrapping.

MODULE 3 Embedded Methods: L_1 Sparsity, Tree MDI & Permutation

The Embedded Paradigm

- Feature selection is built directly into the mathematical objective function.
- Combines the computational speed of filters with the interaction sensitivity of wrappers.
- Occurs naturally during a single training execution.

Pedagogical Coverage

- Regularization geometry: L_1 Lasso diamond vs. L_2 Ridge circle.
- Soft-thresholding operator & Regularization paths.
- Tree Gini Mean Decrease in Impurity (MDI) mechanics.
- Strobl et al. (2007) cardinality bias pitfall.
- Permutation Feature Importance: Model-agnostic truth.

Motivation: The Embedded Paradigm

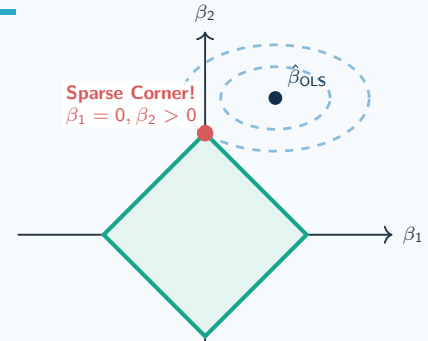
The Best of Both Worlds

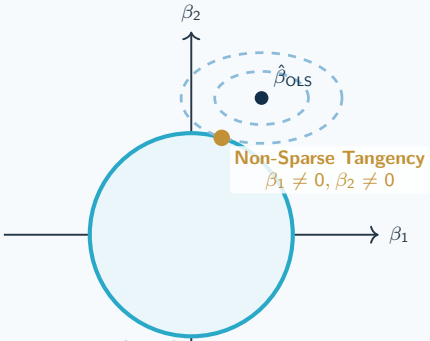
- **Integrated Selection:** Optimization algorithms inherently select features by setting weights to zero or choosing split points.
- **Efficiency of Filters:** Requires only a single model fit (or cross-validated path), avoiding the $O(2^P)$ search loops of wrappers.
- **Interactions of Wrappers:** Fully captures nonlinear interactions and multi-collinear correlations during model optimization.

Three Core Embedded Families

1. L_1 **Regularization (Lasso, ElasticNet):** Penalizes coefficient sum of absolute values, driving non-predictive weights strictly to 0.0.
2. **Tree Impurity (MDI):** Accumulates variance/Gini reduction across tree nodes.
3. **Permutation Importance:** Evaluates validation score drop after shuffling a single feature.

Regularization Geometry: L_1 Lasso vs. L_2 Ridge





Geometric Truth

The sharp corners of the L_1 diamond coincide exactly with the coordinate axes ($\beta_j = 0$). Unpenalized loss ellipses intersect at these corners with high probability, setting coefficients to **exact zero**.

Lasso Sparsity: Mathematical Formulation

The L_1 Regularized Objective

The Lasso (Tibshirani, 1996) solves the constrained optimization:

$$\min_w \frac{1}{2N}\|y - Xw\|_2^2 + \alpha\|w\|_1$$

where $\|w\|_1 = \sum_{j=1}^{P} |w_j|$.

- $\alpha \to 0$: Ordinary Least Squares (all features active).
- $\alpha \to \infty$: Maximum shrinkage (all $w_j = 0$).

Soft-Thresholding Operator

Under orthonormal features ($X^\top X = I$), coordinate descent yields the closed-form thresholding operator:

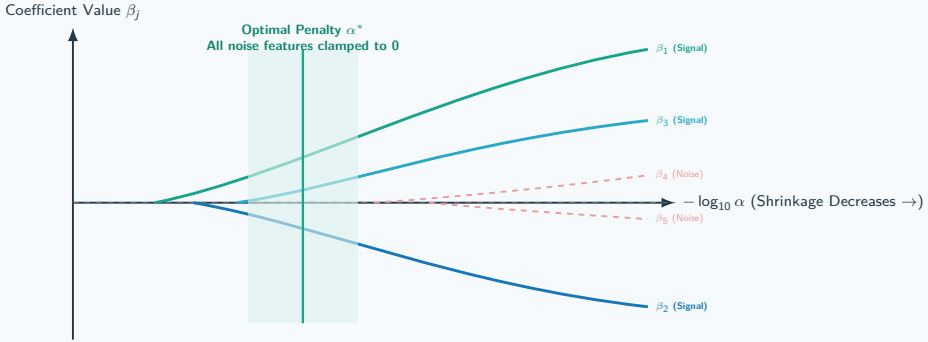
$$\begin{aligned}\hat{w}_j &= \mathcal{S}_\alpha(\hat{w}_j^{\text{OLS}}) \\ &= \text{sign}(\hat{w}_j^{\text{OLS}}) \max\left(0, |\hat{w}_j^{\text{OLS}}| - \alpha\right)\end{aligned}$$

If $|\hat{w}_j^{\text{OLS}}| \leq \alpha$, the coefficient is clamped **strictly to zero**!

Mandatory StandardScaling

Features with larger scales artificially avoid shrinkage. You **must** standardize features before fitting Lasso!

Lasso Sparsity: Regularization Path Analysis



High Penalty ($\alpha > 0.1$)

Extreme shrinkage clamps all coefficients to zero (underfitting, high bias).

Optimal Selection Window ($\alpha \approx 0.03$)

Noise coefficients β_4, β_5 are strictly 0.0, while signals $\beta_1, \beta_2, \beta_3$ are retained!

Lasso Sparsity: Tool and Implementation

Scikit-Learn Implementation

- `LassoCV`: Evaluates coordinate descent path over an automated grid of α values using K-fold cross-validation.
- `SelectFromModel`: Transformer that accepts any fitted model with `coef_` or `feature_importances_`, selecting features where $|\beta_j| >$ `threshold`.

Production Rule

Set `threshold=1e-5` in `SelectFromModel` when using Lasso to extract strictly non-zero features.

```python
from sklearn.linear_model import LassoCV
from sklearn.feature_selection import SelectFromModel
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import numpy as np

# 1. Pipeline with Scaler + LassoCV (auto-tunes alpha via CV)
pipe_lasso = Pipeline([
    ("scaler", StandardScaler()),
    ("lasso", LassoCV(cv=5, random_state=42))
])
pipe_lasso.fit(X, y)

tuned_alpha = pipe_lasso.named_steps["lasso"].alpha_
coefs = pipe_lasso.named_steps["lasso"].coef_

print(f"Optimal Alpha tuned by CV: {tuned_alpha:.4f}")
print("Non-zero coefficients count:", np.sum(coefs != 0))

# 2. Extract selected features via SelectFromModel
selector = SelectFromModel(pipe_lasso.named_steps["lasso"],
                           prefit=True, threshold=1e-5)
print("Selected feature indices :", np.where(selector.get_support())
      [0])
```

Tree Feature Importance: Mean Decrease in Impurity

How Random Forest Computes MDI

In Decision Trees, at every split node t splitting on feature X_j, the impurity decrease is:

$$\Delta I(t) = I(t) - \frac{N_{tL}}{N_t} I(t_L) - \frac{N_{tR}}{N_t} I(t_R)$$

The overall MDI importance of feature j across all B trees is:

$$\text{MDI}_j = \frac{1}{B} \sum_{b=1}^{B} \sum_{t \in T_b : v(t)=j} \frac{N_t}{N} \Delta I(t)$$

Normalized such that $\sum_{j=1}^{P} \text{MDI}_j = 1.0$.

Key Strengths

- Zero additional computational cost (computed during tree construction).
- Captures complex hierarchical non-linearities and threshold rules.

The Catastrophic Flaw

MDI is computed **on training data**. If a tree overfits, pure random noise can exhibit massive MDI!

Tree MDI: Toy Dataset & Hand Calculation

Toy Split: $N = 10$ Samples (5 Pos, 5 Neg)

Root Node t: $I(t) = 1 - (0.5^2 + 0.5^2) = \mathbf{0.50}$.

Candidate Split	Left (t_L)	Right (t_R)	Gini Impurities
$X_1 \leq 3$	4 Pos, 0 Neg ($N_L = 4$)	1 Pos, 5 Neg ($N_R = 6$)	$I_L = 0, I_R = 0.278$
$X_2 \leq 5$	3 Pos, 2 Neg ($N_L = 5$)	2 Pos, 3 Neg ($N_R = 5$)	$I_L = 0.48, I_R = 0.48$

Right Child Gini ($X_1 \leq 3$)

$$I_R = 1 - \left[\left(\tfrac{1}{6}\right)^2 + \left(\tfrac{5}{6}\right)^2\right] = 1 - \tfrac{26}{36} = \tfrac{10}{36} = \mathbf{0.278}$$

Computing Impurity Gain ΔI

Split on X_1:

$$\Delta I(X_1) = 0.50 - 0.167 = \mathbf{0.333}$$

Split on X_2:

$$\Delta I(X_2) = 0.50 - 0.480 = \mathbf{0.020}$$

MDI Attribution

Feature X_1 yields $\Delta I = 0.333$ vs. 0.020 for X_2. X_1 receives **94.3%** of tree importance!

In-Class Drill: Decision Tree Gini MDI Reduction

Problem Statement: Loan Defaults ($N = 8$)

Root node t: 4 Defaults, 4 Paid $\implies I(t) = 0.50$.

- **Split A ($X_1 \leq 30$):** Left ($N_L = 4$): 4 Defaults, 0 Paid $\implies I_L = 0.0$. Right ($N_R = 4$): 0 Defaults, 4 Paid $\implies I_R = 0.0$.
- **Split B ($X_2 \leq 50$):** Left ($N_L = 4$): 3 Defaults, 1 Paid $\implies I_L = 0.375$. Right ($N_R = 4$): 1 Default, 3 Paid $\implies I_R = 0.375$.

Tasks for the Class (2 Minutes)

1. Compute post-split weighted Gini for Split A.
2. Compute post-split weighted Gini for Split B.
3. Compute impurity reduction ΔI_A and ΔI_B.

Step-by-Step Solution

Split A (X_1):

$$I_{\text{split}} = \frac{4}{8}(0.0) + \frac{4}{8}(0.0) = \mathbf{0.0}$$

$$\Delta I(X_1) = 0.50 - 0.0 = \mathbf{0.50} \quad \text{(Pure split!)}$$

Split B (X_2):

$$I_{\text{split}} = \frac{4}{8}(0.375) + \frac{4}{8}(0.375) = \mathbf{0.375}$$

$$\Delta I(X_2) = 0.50 - 0.375 = \mathbf{0.125}$$

Conclusion: Split A is chosen; X_1 earns $4\times$ the importance of X_2 ($\Delta I = 0.50$ vs 0.125).

Tree MDI: Cardinality Bias and High Cardinality

Theoretical Limitation: Strobl et al. (2007)

- **Cardinality Bias:** Continuous features or categorical features with many unique levels (e.g. Transaction ID, Zip Code) offer vastly more candidate split thresholds.
- Even if a high-cardinality column is 100% **pure random Gaussian noise**, a greedy tree can find accidental splits that appear to reduce training impurity!
- **Outcome:** Raw MDI spuriously ranks useless high-cardinality noise at the very top of feature importance rankings!

The Industrial Remedy

Permutation Feature Importance (Breiman, 2001) evaluated on an out-of-sample holdout test partition!

- Shuffling a noise column on test data causes 0.0 drop in test score.
- Completely unmasks the cardinality illusion.

Permutation Feature Importance: Model-Agnostic Truth

Algorithmic Mechanism

1. Fit model $\hat{f}$ on training set $\mathcal{D}_{\text{train}}$.
2. Record baseline validation score S_{base} on untouched validation set $\mathcal{D}_{\text{val}}$ (e.g. AUC or Accuracy).
3. For each feature $j \in \{1, \ldots, P\}$:
   - Permute (randomly shuffle) column X_j^{val}, breaking its link to target y.
   - Re-evaluate validation score $S_{\text{perm},j}$.
   - Importance: $I_j = S_{\text{base}} - S_{\text{perm},j}$.
4. Repeat n times to obtain mean importance and standard error.

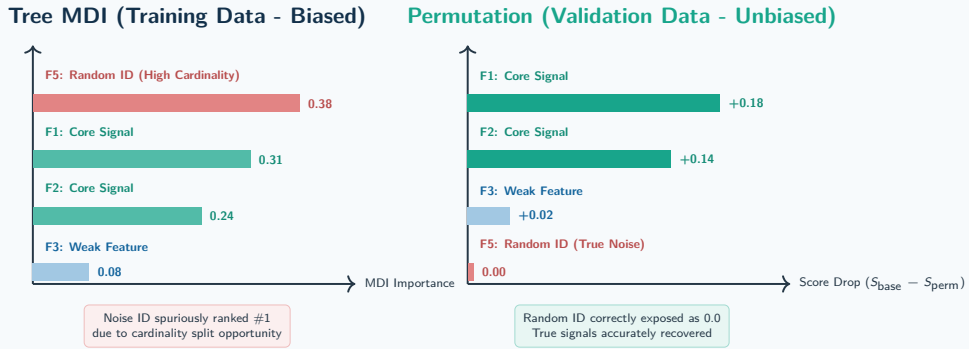
Why Permutation is Superior

- **Zero Cardinality Bias:** Evaluated on out-of-fold data.
- 100% **Model-Agnostic:** Works identically for linear models, trees, neural nets, or stacked ensembles.
- Reflects true real-world degradation if the feature disappears.

Negative Importance?

If $I_j < 0$, permuting the feature actually *improved* the score. That feature was adding harmful noise!

Permutation Importance: Empirical Comparison vs. MDI



Executive Principle

Never deploy feature selection based purely on training MDI. Always audit with out-of-fold Permutation Importance to prevent deploying noise.

Hands-on Lab 4: Tree MDI vs Permutation Importance

Lab Diagnostic Setup

- Inject a synthetic column with 10,000 distinct random floating numbers (`random_cardinality`).
- Fit Random Forest on training data.
- Compare raw `feature_importances_` against `permutation_importance` on the test holdout.

```python
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
import numpy as np

# 1. Load data and inject high-cardinality random noise column
data = load_breast_cancer()
X = np.c_[data.data, np.random.randn(len(data.data))] # Col 30 is
        noise
y = data.target

X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=42)
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(
        X_tr, y_tr)

# 2. Compare MDI vs Permutation on Test Set
print("MDI Noise Rank:", np.argsort(rf.feature_importances_)
        [::-1][:5])

perm = permutation_importance(rf, X_te, y_te, n_repeats=10,
        random_state=42)
print("Permutation Noise Score:", np.round(perm.importances_mean[-1],
        4))
# Permutation mean drops strictly to <= 0.0!
```

Module 3 Synthesis: Embedded Methods Decision Matrix

Embedded Technique	Model Family	Metric Basis	Speed	Multicollinearity Behavior	Primary Production Hazard
Lasso (L_1 Penalty)	Linear / Logistic	Sparsity via $\lvert\beta_j\rvert = 0$	Ultra-fast	Arbitrarily picks 1 from correlated set	Fails if features are unscaled
ElasticNet ($L_1 + L_2$)	Linear / Logistic	Grouped sparsity	Ultra-fast	Retains groups of correlated features	Extra hyperparameter (`l1_ratio`)
Tree MDI (Gini)	Random Forest / GBDT	Impurity reduction ΔI	Zero cost	Splits importance among correlated trees	Severe bias toward high cardinality
Permutation Importance	100% Model-Agnostic	Out-of-sample score drop	Moderate	Correlated features dilute importance	Requires separate validation split

Executive Recommendation

Use **ElasticNet** for high-dimensional linear problems with collinearity; use **Random Forest + Permutation Importance** for complex non-linear enterprise domains.

MODULE 4 Advanced Paradigms, Hybrid Funnels & Production Pipelines

Production Realities

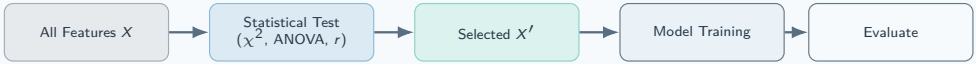
- In real financial systems, no single selection paradigm is optimal in isolation.
- Collinearity, high cardinality, and data leakage destroy models if unaddressed.
- Industrial ML requires multi-stage hybrid funnels.

Pedagogical Coverage

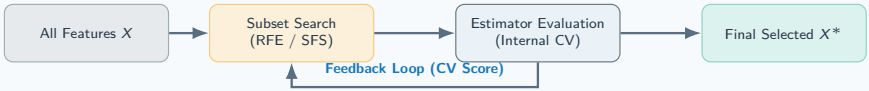
- 3 Paradigms comparison: Architectural topology.
- Collinearity & High-cardinality survival strategies.
- Mathematical proof of selection leakage (Freedman's Paradox).
- Encapsulation via `Pipeline` & `ColumnTransformer`.
- Grand Multi-Stage Funnel: Filter $\rightarrow$ Embedded $\rightarrow$ Wrapper.

The 3 Paradigms in Contrast: Architectural Topology

1. Filter Paradigm



2. Wrapper Paradigm



3. Embedded Paradigm



Filters:
Fast independent ranking, zero interaction feedback.

Wrappers:
Iterative model loop, highly accurate, slow.

Embedded:
Unified optimization, optimal balance of speed and synergy.

Collinearity Effects Across Selection Paradigms

How Multicollinearity Distorts Selection ($\rho > 0.90$)

When features X_1 and X_2 are highly correlated:

- **In Filter Methods:** Both achieve nearly identical high test statistics; both are selected, causing redundancy!
- **In Lasso (L_1):** Arbitrarily picks one and sets the other to zero based on micro-fluctuations in sample noise.
- **In Random Forest (MDI):** Tree splits divide between X_1 and X_2, diluting the importance of both by $\approx 50\%$!

The Industrial Collinearity Protocol

1. **Correlation Matrix Triage:** Compute Pearson/Spearman matrix before modeling.
2. **Threshold Pruning:** When $|\rho(X_i, X_j)| > 0.85$, drop the feature with lower individual target correlation.
3. **ElasticNet Alternative:** If linear models are required, use ElasticNet (`l1_ratio=0.5`) to retain grouped correlated features together.

High Cardinality Across Feature Selection Paradigms

The Threat to Selection Integrity

Categorical features with high unique counts (Zip Code, Merchant ID, IP Subnet):

- **Chi-Squared (χ^2):** Degrees of freedom $df = (r-1)(c-1)$ explode. Noise fluctuations across hundreds of cells artificially inflate χ^2 test statistics.
- **Tree Models (MDI):** Excessive split opportunities cause trees to overfit, artificially inflating training impurity decrease.

Standardized Defense Protocol

1. **Frequency Capping:** Group categories with $< 1\%$ frequency into an `"Other"` bucket.
2. **Target Encoding with Smoothing:** Replace categories with smoothed out-of-fold target averages (Micci-Barreca, 2001).
3. **Permutation Audit:** Always evaluate importance on an out-of-fold test set.

Production Guardrail: Zero Selection Data Leakage

The #1 Flaw: Selection Before Splitting

Catastrophic Error:

$$X_{\text{selected}} = \texttt{SelectKBest}(X_{\text{all}}, y_{\text{all}}) \implies \text{split}(X_{\text{selected}})$$

- Running feature selection on the whole dataset **leaks test labels into feature choice**.
- In datasets with $10,000$ pure noise features, chance correlations guarantee the top 50 features correlate strongly with y.
- Yields fake 95% cross-validation accuracy on pure noise!

The Universal Remedy: Scikit-Learn Pipeline

- Always call `train_test_split` **FIRST**.
- Wrap feature selection inside a `Pipeline`.
- During cross-validation, selection is re-fitted independently on each training fold!

Audit Check

If training accuracy is 98% but holdout test accuracy is 55%, check for selection leakage before splitting.

Mathematical Proof: Selection Leakage on Pure Noise

Freedman's Paradox (Freedman, 1983)

Let $X \sim \mathcal{N}(0, I)$ be $N = 100$ samples of $P = 1,000$ independent Gaussian noise features. Let $y \sim$ Bernoulli(0.5) be random coin flips.

- True population correlation: $\rho(X_j, y) = 0 \quad (\forall j)$.
- By Extreme Value Theory, the sample maximum $|r_{\max}|$ across P independent noise features follows:

$$\mathbb{E}\left[\max_{j=1,\ldots,P} |r_j|\right] \approx \sqrt{\frac{2 \ln P}{N}}$$

Theoretical Derivation

For $N = 100, P = 1,000$:

$$\mathbb{E}[|r_{\max}|] \approx \sqrt{\frac{2\ln(1000)}{100}} = \sqrt{\frac{13.816}{100}}$$
$$\approx \sqrt{0.1382} \approx \mathbf{0.372}$$

Selecting the top 50 features from 1,000 noise columns yields sample correlations $r \approx 0.35$–0.40!

The Disaster

Regressing y on these 50 leaked noise features yields $R^2 \approx 0.50$ and an ANOVA F-test $p < 10^{-6}$ on pure noise!

In-Class Drill: Selection Leakage & Freedman's Paradox

Problem Statement: Noise Screening ($N = 50, P = 200$)

Pure Gaussian noise matrix $X_{50\times 200} \sim \mathcal{N}(0,1)$ and random coin-flip labels $y \in \{0,1\}$.

- True population correlation: $\rho_j \equiv 0.0, \forall j$.
- Extreme Value Theory bound:

$$\mathbb{E}[|r_{\max}|] \approx \sqrt{\frac{2\ln P}{N}}$$

Tasks for the Class (2 Minutes)

1. Compute ln(200) (Note: $\ln 200 \approx 5.298$).
2. Compute $\mathbb{E}[|r_{\max}|]$ for $N = 50, P = 200$.
3. What happens if we select the top 10 features before cross-validation?

Step-by-Step Solution

Step 1: Expected Maximum Correlation

$$\mathbb{E}[|r_{\max}|] \approx \sqrt{\frac{2 \times 5.298}{50}} = \sqrt{\frac{10.596}{50}}$$
$$\approx \sqrt{0.2119} \approx \mathbf{0.460}$$

Step 2: Spurious Association

- By pure chance, the best noise feature has sample correlation $r \approx 0.46$!
- Converting to t-statistic ($df = 48$):

$$t = \frac{0.46\sqrt{48}}{\sqrt{1 - 0.46^2}} \approx 3.58 \implies p = 0.0008!$$

- Feature selection before split falsely validates noise as a statistically significant predictor!

Automated Production Solution: Scikit-Learn Pipeline

The Pipeline Solution

- Split data into training and test partitions **first**.
- Wrap `SelectKBest` and model into a single `Pipeline`.
- When `cross_val_score` runs, selection is isolated inside each CV fold!

Zero Leakage Guarantee

`Pipeline.fit(X_train, y_train)` calculates selection scores exclusively from training samples. The test set remains strictly unseen.

```python
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.ensemble import RandomForestClassifier

# 1. Split FIRST - strictly isolate test data
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
       random_state=42)

# 2. Encapsulate Selection inside Pipeline
pipe = Pipeline([
    ("filter", SelectKBest(score_func=f_classif, k=5)),
    ("clf", RandomForestClassifier(n_estimators=100, random_state=42))
])

# 3. Leakage-free Cross-Validation
cv_scores = cross_val_score(pipe, X_tr, y_tr, cv=5, scoring="accuracy"
       )
print(f"5-Fold CV Accuracy: {cv_scores.mean():.4f} +/- {cv_scores.std
       ():.4f}")

# 4. Final Evaluation on Untouched Test Holdout
pipe.fit(X_tr, y_tr)
print(f"Test Holdout Score: {pipe.score(X_te, y_te):.4f}")
```

Heterogeneous Selection with ColumnTransformer

Heterogeneous Routing

Real enterprise datasets combine numbers and categories:

- **Numeric Columns:** Impute $\rightarrow$ Scale $\rightarrow$ ANOVA F-test.
- **Categorical Columns:** One-Hot Encode $\rightarrow$ χ^2 Test.

Use `ColumnTransformer` to execute parallel selection pipelines!

```python
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import SelectPercentile, chi2, f_classif
from sklearn.preprocessing import MinMaxScaler, OneHotEncoder

num_cols = ["age", "income", "credit_score"]
cat_cols = ["education", "home_ownership"]

# Apply Chi2 to categorical and ANOVA to numerical
preprocessor = ColumnTransformer(transformers=[
    ("num_select", Pipeline([
        ("scale", MinMaxScaler()),
        ("anova", SelectPercentile(score_func=f_classif, percentile=60))
    ]), num_cols),
    ("cat_select", Pipeline([
        ("ohe", OneHotEncoder(handle_unknown="ignore")),
        ("chi2", SelectPercentile(score_func=chi2, percentile=50))
    ]), cat_cols)
])

# Full model pipeline
full_pipe = Pipeline([
    ("prep", preprocessor),
    ("clf", RandomForestClassifier(random_state=42))
])
```

Hands-on Lab 5: Grand End-to-End Selection Pipeline

Grand 3-Stage Architecture

1. **Stage 1 (Unsupervised Filter):** `VarianceThreshold` eliminates constants.
2. **Stage 2 (Embedded Sparsity):** `SelectFromModel(Lasso)` removes redundant collinear features.
3. **Stage 3 (High-Capacity Estimator):** Gradient Boosting trained on clean, sparse subset.

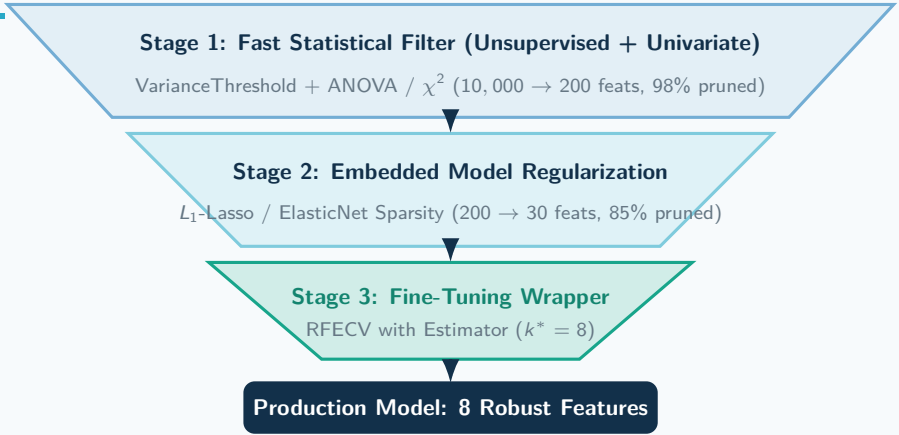
Memory Management: Funneling reduces P before complex trees, guarding distributed workers against driver OOM.

```python
from sklearn.pipeline import Pipeline
from sklearn.feature_selection import VarianceThreshold, SelectFromModel
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report

# Grand 3-Stage Pipeline:
grand_pipe = Pipeline([
    ("zero_var", VarianceThreshold(threshold=0.0)),
    ("scaler", StandardScaler()),
    ("lasso_embed", SelectFromModel(
        LogisticRegression(penalty="l1", solver="liblinear", C=0.2,
                random_state=42)
    )),
    ("gbdt", GradientBoostingClassifier(n_estimators=100, random_state=42))
])

grand_pipe.fit(X_tr, y_tr)
preds = grand_pipe.predict(X_te)
print(classification_report(y_te, preds))
```

Hybrid Feature Selection: The Multi-Stage Funnel



Production Funnel Principle

Never run wrappers on raw data! Combine fast filter screening to eliminate noise, embedded methods for structured pruning, and targeted wrappers for final subset refinement.

Module 4 Synthesis: Master Selection Strategy Matrix

Paradigm	Compute Cost	Model-Agnostic?	Interaction Handling	Overfitting Risk	Key Scikit-Learn APIs
Filter	Ultra-Low $O(NP)$	Yes (100%)	None (Univariate)	Very Low	`SelectKBest`, `chi2`, `f_classif`
Wrapper	Very High $O(2^P)$	Yes (SFS) / Semi (RFE)	Complete multi-way	High (needs strict CV)	`RFE`, `RFECV`, `SFS`
Embedded	Moderate (1 fit)	No (Model-Specific)	Model-dependent	Moderate	`LassoCV`, `SelectFromModel`, MDI
Hybrid Funnel	Optimized	Semi-Agnostic	Staged multi-level	Minimal (encapsulated)	`Pipeline` + Filter $\rightarrow$ Lasso $\rightarrow$ RFECV

The Golden Engineering Rule

Never deploy raw feature selection outside an encapsulated `Pipeline`. Wrap all selectors and scalers to prevent data leakage and guarantee reproducibility.

Summary & Actionable Feature Selection Checklist

Checklist: Modules 1 & 2

- ☐ **Match Filter to Data Types:** χ^2 for categorical/count data ($X \geq 0$); ANOVA F-test for continuous numeric inputs.
- ☐ **Collinearity Screening:** Drop redundant features with $|\rho| > 0.85$ before running complex models.
- ☐ **Wrapper Pre-screening:** Never run RFECV directly on $P > 50$ features; pre-screen with filters first.

Checklist: Modules 3 & 4

- ☐ **Lasso Standardization:** Always standard-scale features before computing L_1 penalties.
- ☐ **Permutation Importance Audit:** Validate tree MDI with out-of-fold permutation scores to defeat cardinality bias.
- ☐ **Zero Leakage Pipeline:** Always split train/test before selection; encapsulate all steps inside Scikit-Learn `Pipeline`.

Thank You!

Questions & Answers (Q&A)

Faculty of Mathematical Economics | National Economics University